Sub Queries in Sql in Telugu | DBMS Tutorial | MySQL

Believer 01

0:10 / 12:10

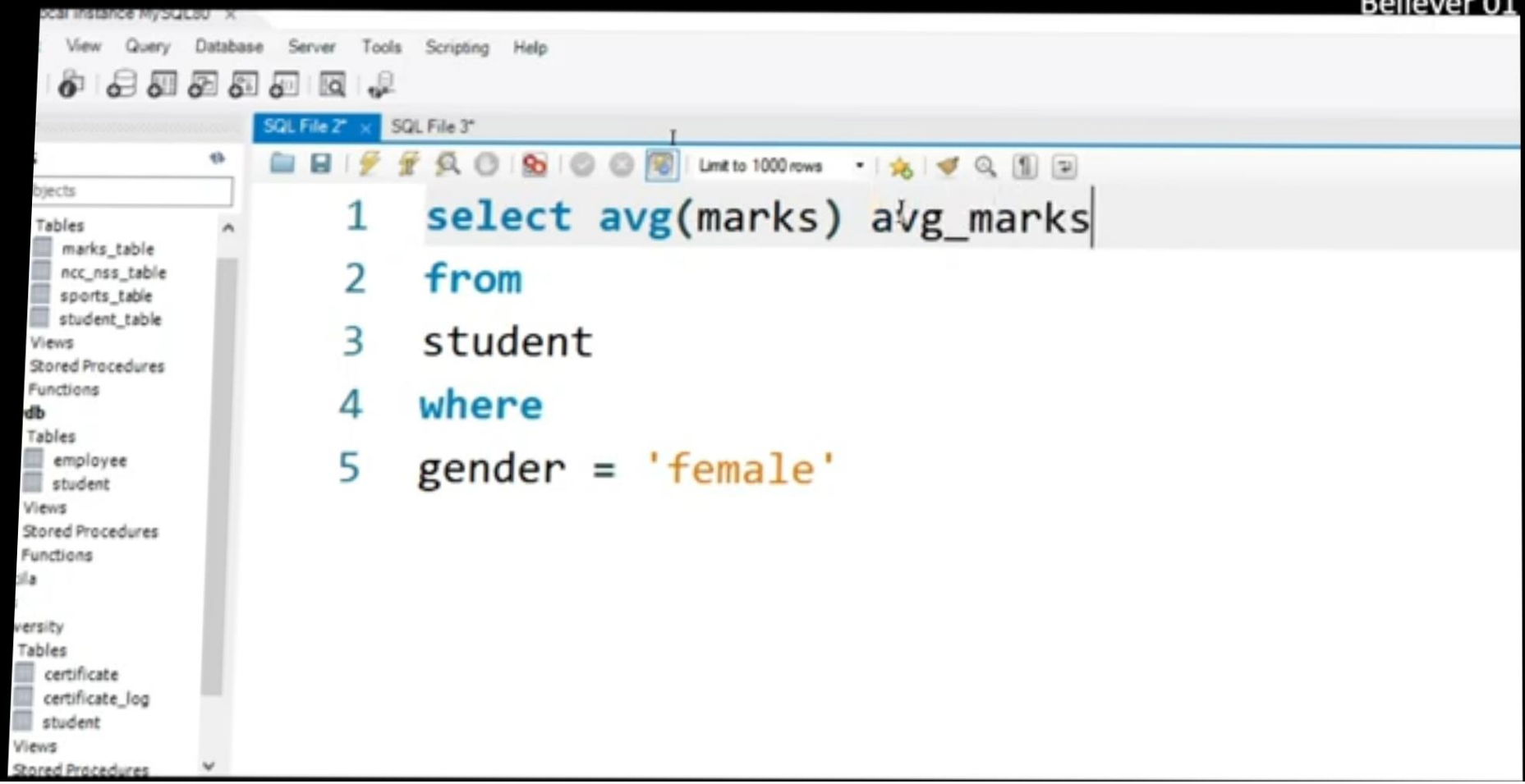
More videos

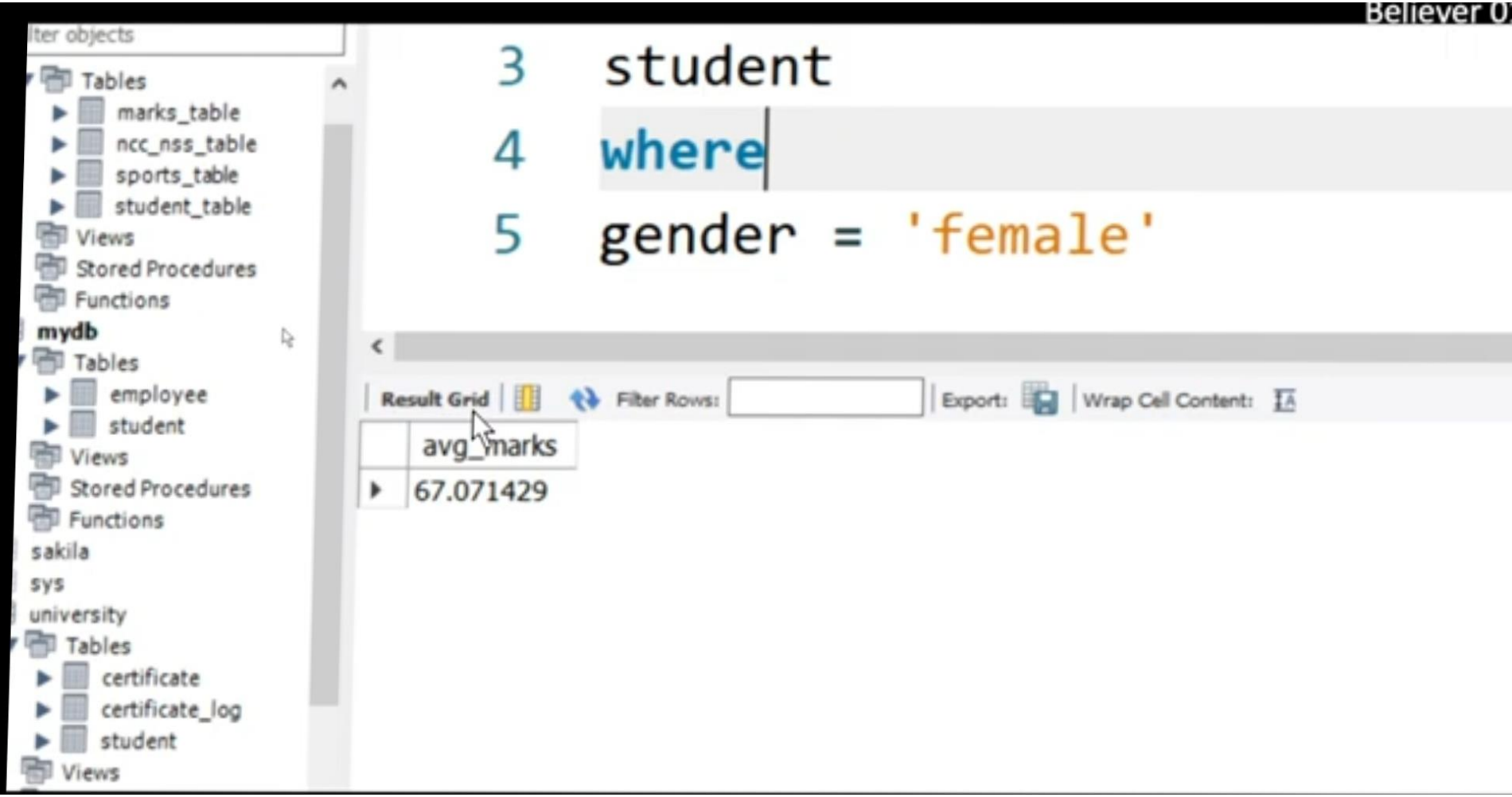
Tap or swipe up to see all

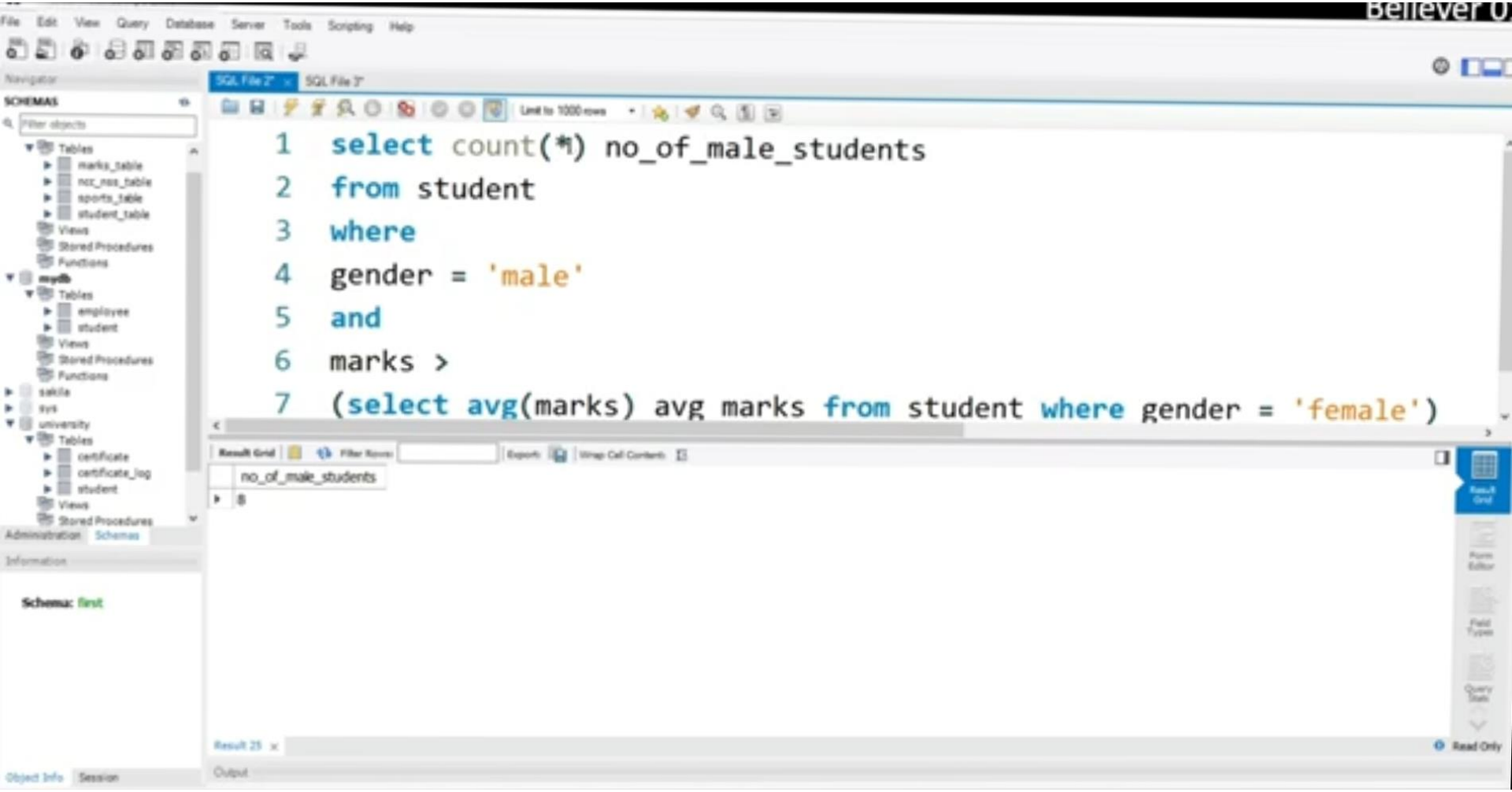


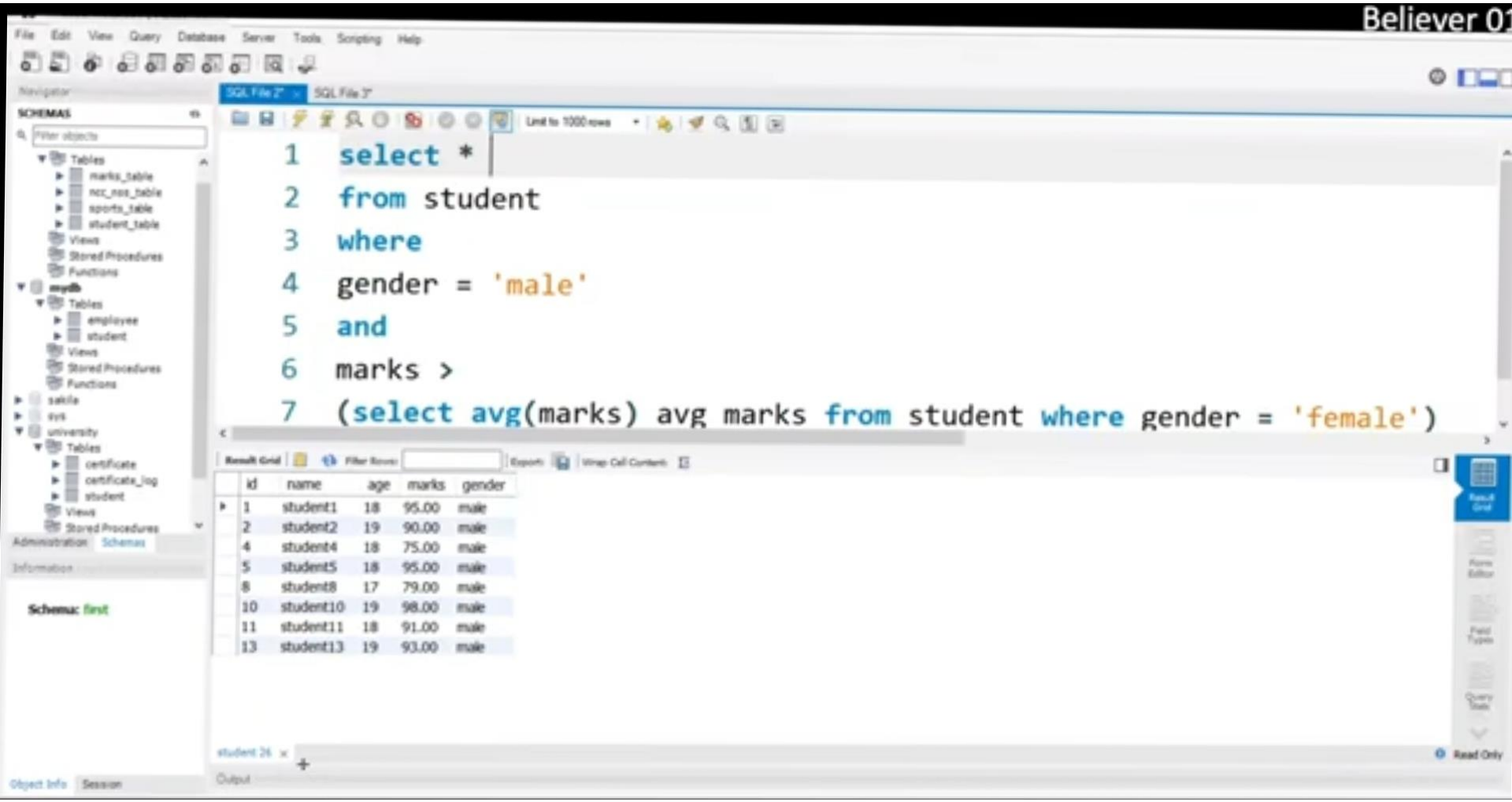
```
1
2 sql subqueries
3
4 1) normal subqueries
5 2) co-related subqueries
```

```
 7
 8 i want find to number of
 9 male students who have marks
10 more than average marks of
11 female students
12
13
14
```


View Query Database Server Tools Scripting Help
SQL File 2*
SQL File 3*
Limit to 1000 rows
Tables
marks_table
ncc_nss_table
sports_table
student_table
Views
Stored Procedures
Functions
db
Tables
employee
student
Views
Stored Procedures
Functions
Tables
certificate
certificate_log
student
Views
1 select avg(marks) avg_marks
2 from
3 student
4 where
5 gender = 'female'


Believer 0
lter objects
Tables
marks_table
ncc_nss_table
sports_table
student_table
Views
Stored Procedures
Functions
mydb
Tables
employee
student
Views
Stored Procedures
Functions
sakila
sys
university
Tables
certificate
certificate_log
student
Views
3 student
4 where
5 gender = 'female'
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
avg_marks
67.071429


Believer 0
File Edit View Query Database Server Tools Scripting Help
SCHEMAS
SQL File 2*
SQL File 3*
Limit to 1000 rows
1 select count(*) no_of_male_students
2 from student
3 where
4 gender = 'male'
5 and
6 marks >
7 (select avg(marks) avg_marks from student where gender = 'female')
Result Grid
no_of_male_students
8
Schema: first
Result 25
Read Only
Output


Believer 01
File Edit View Query Database Server Tools Scripting Help
SCHEMAS
Tables
marks_table
ncc_nss_table
sports_table
student_table
Views
Stored Procedures
Functions
mydb
Tables
employee
student
Views
Stored Procedures
Functions
sakila
sys
university
Tables
certificate
certificate_log
student
Views
Stored Procedures
Administration Schemas
Schema: first
Object Info Session
SQL File 2* SQL File 3*
Limit to 1000 rows
1 select *
2 from student
3 where
4 gender = 'male'
5 and
6 marks >
7 (select avg(marks) avg_marks from student where gender = 'female')
Result Grid
id name age marks gender
1 student1 18 95.00 male
2 student2 19 90.00 male
4 student4 18 75.00 male
5 student5 18 95.00 male
8 student8 17 79.00 male
10 student10 19 98.00 male
11 student11 18 91.00 male
13 student13 19 93.00 male
student 26
Read Only
Output

JOINS

→ Combine rows from 2 or more tables based on a related column

Customers

id	cust_name	designation
1	abc	A
2	pqr	B
3	mno	C

orders

or_id	amount	cust_id
601	10000	1
602	2000	4
603	1500	2

① INNER JOIN / JOIN.

SELECT orders.or_id, Customers.cust_name

Combine rows from 2 or more tables based on a related column

Customers

id	cust_name	designation
1	abc	A
2	pqr	B
3	mno	C

orders

or_id	amount	cust_id
601	10000	1
602	2000	4
603	1500	2

① INNER JOIN / JOIN.

```
SELECT orders.or_id, Customers.cust_name
FROM ORDERS
INNER JOIN CUSTOMERS ON orders.cust_id =
Customers.id ;
```

⇒

or_id	cust_name
601	abc
603	pqr



FROM ORDERS
INNER JOIN CUSTOMERS ON orders.cust_id = Customers.id ;

⇒

or_id	cust_name
601	abc
603	pqr



② LEFT JOIN



SELECT Orders.or_id, Customers.cust_name
FROM ORDERS
LEFT JOIN CUSTOMERS ON Orders.cust_id = Customers.id;

or_id	cust_name
601	abc
602	null
603	pqr

JOINS

→ Combine rows from 2 or more tables based on a related column

Customers

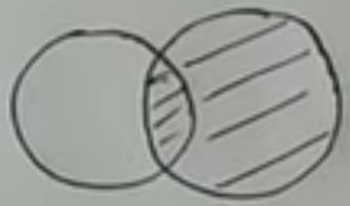
id	cust_name	designation
1	abc	A
2	pqr	B
3	mno	C

orders

or_id	amount	cust_id
601	10000	1
602	2000	4
603	1500	2

① INNER JOIN / JOIN.

③ RIGHT JOIN



Or_id	Cust_name
601	abc
603	pqr
null	mno

④ FUL OUTER JOIN



601	abc
603	pqr
null	mno

④ FULL OUTER JOIN



or_id	cust_name
601	abc
602	null
603	pqr
null	mno

⑤ SELF JOIN

null	mno

(5) SELF JOIN

→ one table considered as 2 tables.
→ Join table to itself.

emp_id	name	manager_id	DOJ
121	abc	321	...
321	pqr	986	...
421	mno	876	...

```
SELECT DISTINCT E.name
FROM Employee E
INNER JOIN Employee M
ON E.emp_id = M.manger_id ;
```

null	mno

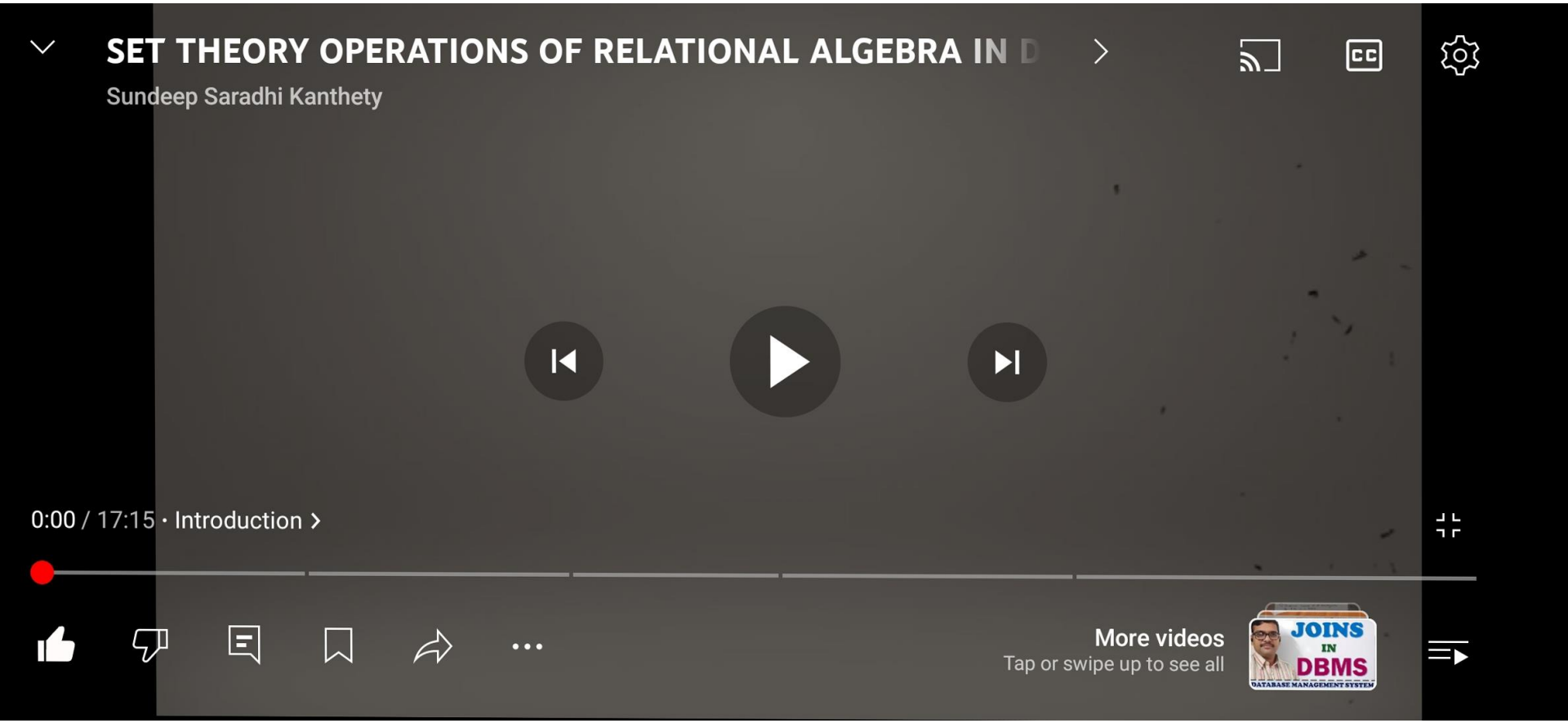
(5) SELF JOIN

→ one table considered as 2 tables.
→ Join table to itself.

emp_id	name	manager_id	DOJ
121	abc	321	...
321	pqr	986	...
421	mno	876	...

```
SELECT DISTINCT E.name
FROM Employee E
INNER JOIN Employee M
ON E.emp_id = M.manger_id ;
```

output :- pqr


SET THEORY OPERATIONS OF RELATIONAL ALGEBRA IN D
Sundeep Saradhi Kanthety
0:00 / 17:15 · Introduction
More videos
Tap or swipe up to see all
JOINS IN DBMS
DATABASE MANAGEMENT SYSTEM

Set Theory Operations in DBMS

* Union (∪)
* Intersection (∩)
* Set Difference / Minus (−)
* Cross Product (x)

Input → 2 Relations

Output → 1 Relation

Set Theory Operations in DBMS

Union (∪)

* Resultant Relation - R ∪ S
* R ∪ S → All the tuples of R and S
* Automatically it will remove duplicates

3 Rows

Cid	Cname
101	C++
102	DBMS
103	Python

R

3 Rows

Cid	Cname
101	C++
104	Java
105	C

S

Cid	Cname
101	C++
102	DBMS
103	Python
104	Java
105	C

R ∪ S

Set Theory Operations in DBMS

Intersection (∩)

* Resultant Relation - $R \cap S$
* $R \cap S$ → All the tuples that are common in both R & S
* Automatically it will remove duplicates

3 Rows

Cid	Cname
101	C++
102	DBMS
103	Python

R

3 Rows

Cid	Cname
101	C++
104	Java
105	C

S

Cid	Cname
101	C++

$R \cap S$

Set Theory Operations in DBMS

Set Difference / Minus (-)

* Resultant Relation → R − S
* R − S → All the tuples of Relation R but not in Relation S

R

Cid	Cname
101	C++
102	DBMS
103	Python

S

Cid	Cname
101	C++
104	Java
105	C

R − S

Cid	Cname
102	DBMS
103	Python

S − R

Cid	Cname
104	Java
105	C

Set Theory Operations in DBMS

Cross Product (X)

Resultant Relation → R X S

R X S → Every tuple of R is associated with Every tuple of S

Cid	Cname
101	C++
102	DBMS
103	Python

R

3 Rows

Sid	Sname
S1	Sundeep
S2	Saradhi

S

2 Rows

R X S → (3 x 2) Rows

R X S

Cid	Cname	Sid	Sname
101	C++	S1	Sundeep
101	C++	S2	Saradhi
102	DBMS	S1	Sundeep
102	DBMS	S2	Saradhi
103	Python	S1	Sundeep
103	Python	S2	Saradhi

① Increase income of all employees by 5% in a table.

→ UPDATE Employees SET income = income + (income * 5.0/100.0);

② Find names of employee starting with 'A'.

→ SELECT first_name FROM Employees where first_name LIKE 'A%'

'A%' → null/anynos alphabets

③ find number of employees working in department 'ABC'.

(4) Print details of employees whose fir
ends with 'A' and contains 6 Alp

→ SELECT * FROM EMPLOYEES
WHERE FIRST_NAME LIKE '_____A'

anyone alphabet

(5) Print details of employees whose salary lies between 10 000 and 50 000.

→ SELECT * FROM Employees
WHERE Salary BETWEEN 10 000
AND 50 000;

L.L

U.Limit

(3) find number of employees working in department 'ABC'.

→ SELECT count(*) FROM Employees
where department_name = 'ABC';

(4) Print details of employees whose first name ends with 'A' and contains 6 Alphabets.

→ SELECT * FROM EMPLOYEES
WHERE FIRST_NAME LIKE '_____A';

(5) Print details of employees whose salary lies between 10 000 and 50

4. Create empty table with same structure as some other table.

```
SELECT * INTO Newtable FROM oldtable
WHERE 1=0;
```

5. Fetch common records bet^n 2 tables.

```
SELECT * FROM Table1
INTERSECT
SELECT * FROM Table2
```

1. fetch only the first name from full name column.

```
SELECT substring (fullname, 0,
        Charindex (' ', fullname))
FROM employees;
```

↳ space is returned

index of

2. fetch duplicate records from a table.

```
SELECT empid, dept, count(*)
FROM employees
GROUP BY empid, dept
HAVING count(*) > 1
```

↳ one

3. Remove duplicates

```
DELETE FROM employees
WHERE empid IN ( SELECT empid from emp
                 GROUP BY dept
```

2. fetch duplicate records

```
SELECT emp_id, dept, count(*)
FROM employees
GROUP BY emp_id, dept
HAVING count(*) > 1
```

3. Remove duplicates

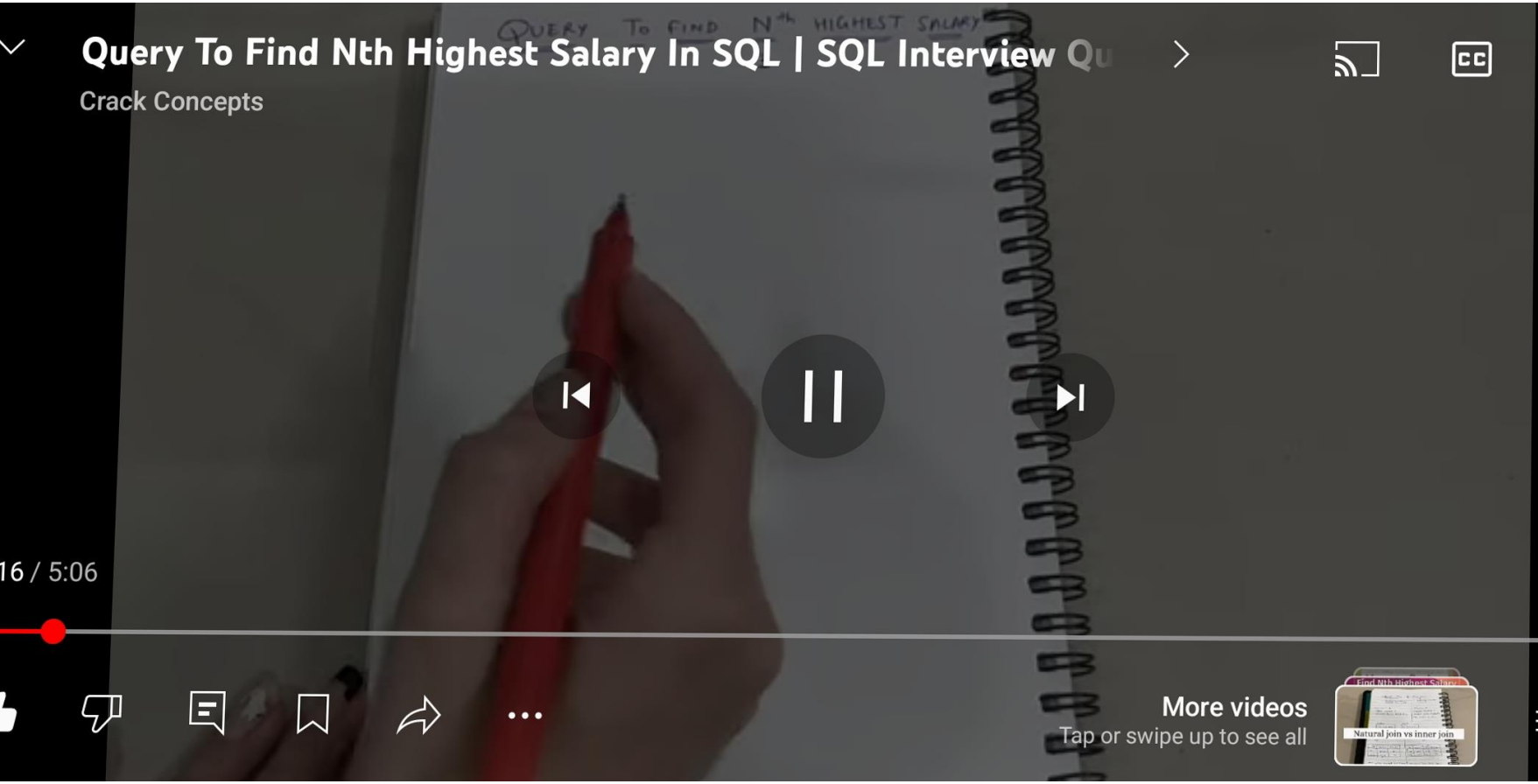
```
DELETE FROM employees
WHERE emp_id IN ( SELECT emp_id FROM emp
                  GROUP BY dept
                  HAVING count(*) > 1 );
```

K/W

4. Create empty table with same structure as some other table.

```
SELECT * INTO Newtable FROM oldtable
WHERE 1=0;
```

→ condition false means no row stored in table.


Query To Find Nth Highest Salary In SQL | SQL Interview Qu
Crack Concepts
16 / 5:06
More videos
Tap or swipe up to see all
Natural join vs inner join

```
SELECT TOP 1 SAL
FROM
(SELECT TOP N SAL
 FROM emps ORDER
 BY SAL DESC..)
ORDER BY SAL ASC
```

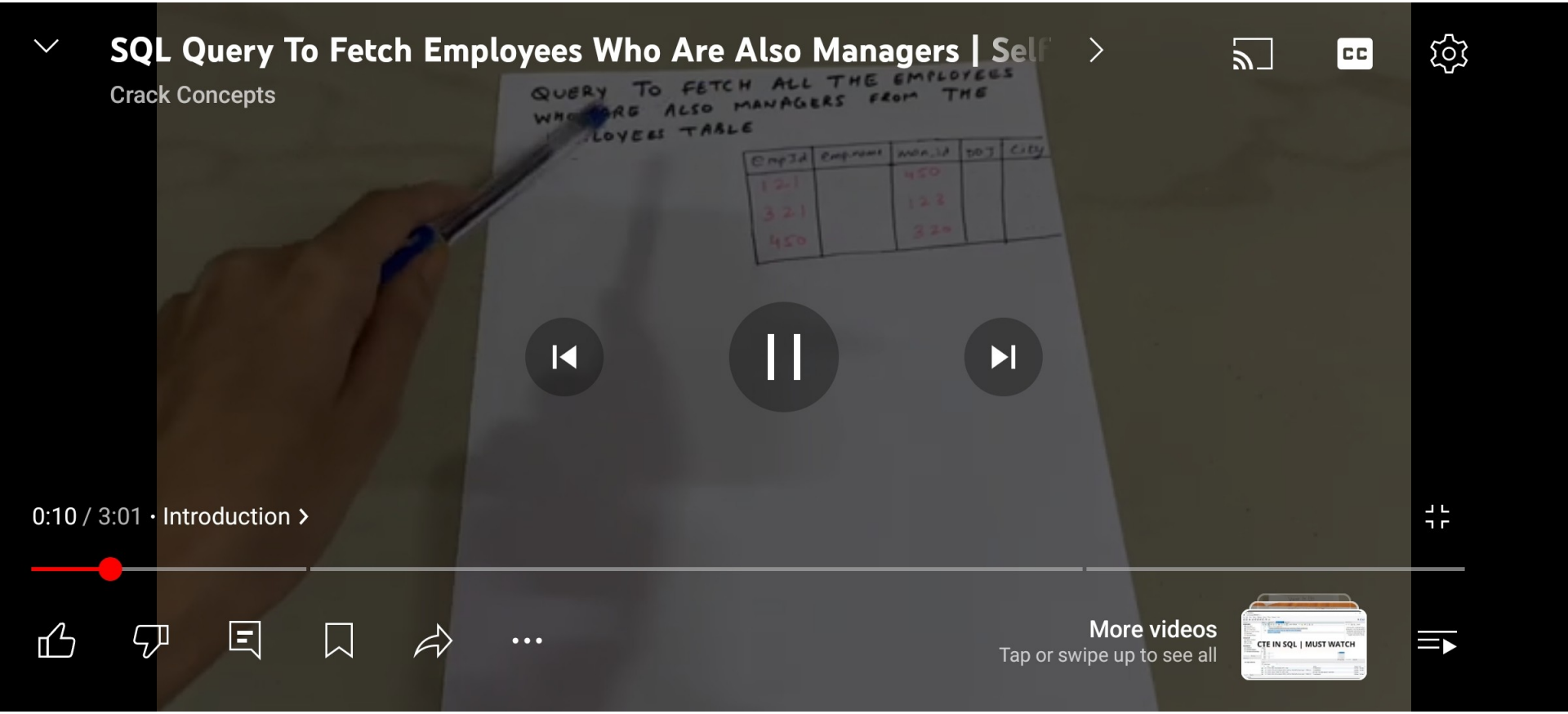
id	SAL
1	1000
2	2000
3	5000
4	3000
5	6000

=> 3000

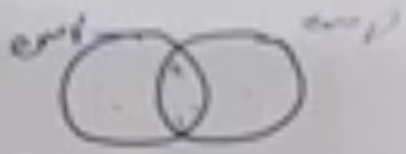
6000
5000
3000
2000
1000

```
SELECT TOP 1 SAL
FROM
(SELECT TOP N SAL
 FROM emps
 ORDER BY SAL DESC)
ORDER BY SAL ASC;
```

3000
5000
6000


SQL Query To Fetch Employees Who Are Also Managers | Self
Crack Concepts
QUERY TO FETCH ALL THE EMPLOYEES
ALSO MANAGERS FROM THE
TABLE
0:10 / 3:01 · Introduction
More videos
Tap or swipe up to see all
CTE IN SQL | MUST WATCH

QUERY TO FETCH ALL THE EMPLOYEES WHO ARE ALSO MANAGERS FROM THE EMPLOYEES TABLE

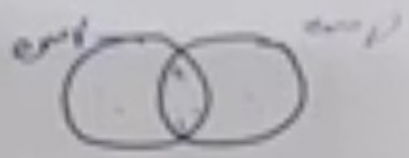


SELF JOIN

EmpId	Empname	Mgr_id	DOJ	City
121		450		
321		123		
450		320		

```
SELECT Distinct emp-id,
FROM employees E
INNER JOIN employees M
ON E.empid = M.empid
```

QUERY TO FETCH ALL THE EMPLOYEES WHO ARE ALSO MANAGERS FROM THE EMPLOYEES TABLE



SELF JOIN

EmpId	Empname	manid	DOJ	City
121	..	450	..	..
321	..	123	..	..
450	..	320	..	..

```
SELECT Distinct emp-id,
FROM employees E
INNER JOIN employees M
ON E.empid = M.[redacted]
```

mangd

what is the simple definition for sql left join

A **SQL LEFT JOIN** is a type of join that returns all the records from the left table and the matched records from the right table. If there is no match, the result is NULL from the right table. It's useful when you want to see all the data from one table, even if there's no corresponding data in the other table.

what is innter join definition in brief

An **SQL INNER JOIN** is a type of join that returns only the rows where there is a match in both tables. If there are no matching records in the other table, those rows are not included in the result. It's used when you want to retrieve only the data that is common between the two tables based on the join condition.

Message ChatGPT

SALARY
2000.00
1500.00
2000.00
1625.00
2125.00
1125.00
10000.00

ke any other statements mentioned

Natural Join V/s Inner join.

Natural Join
↓
No need to mention common row in both.

Inner join
↓
explicitly mention

```
SELECT *
FROM student S
NATURAL JOIN Marks m;
```

```
SELECT *
FROM Student S
INNER JOIN Marks M
ON S.id = m.id;
```

Student

id	name
101	abc
102	pqr
103	xyz

Marks

id	Marks
102	100
103	70
104	80

o/p:

id	name	Marks
102	pqr	100
103	xyz	70

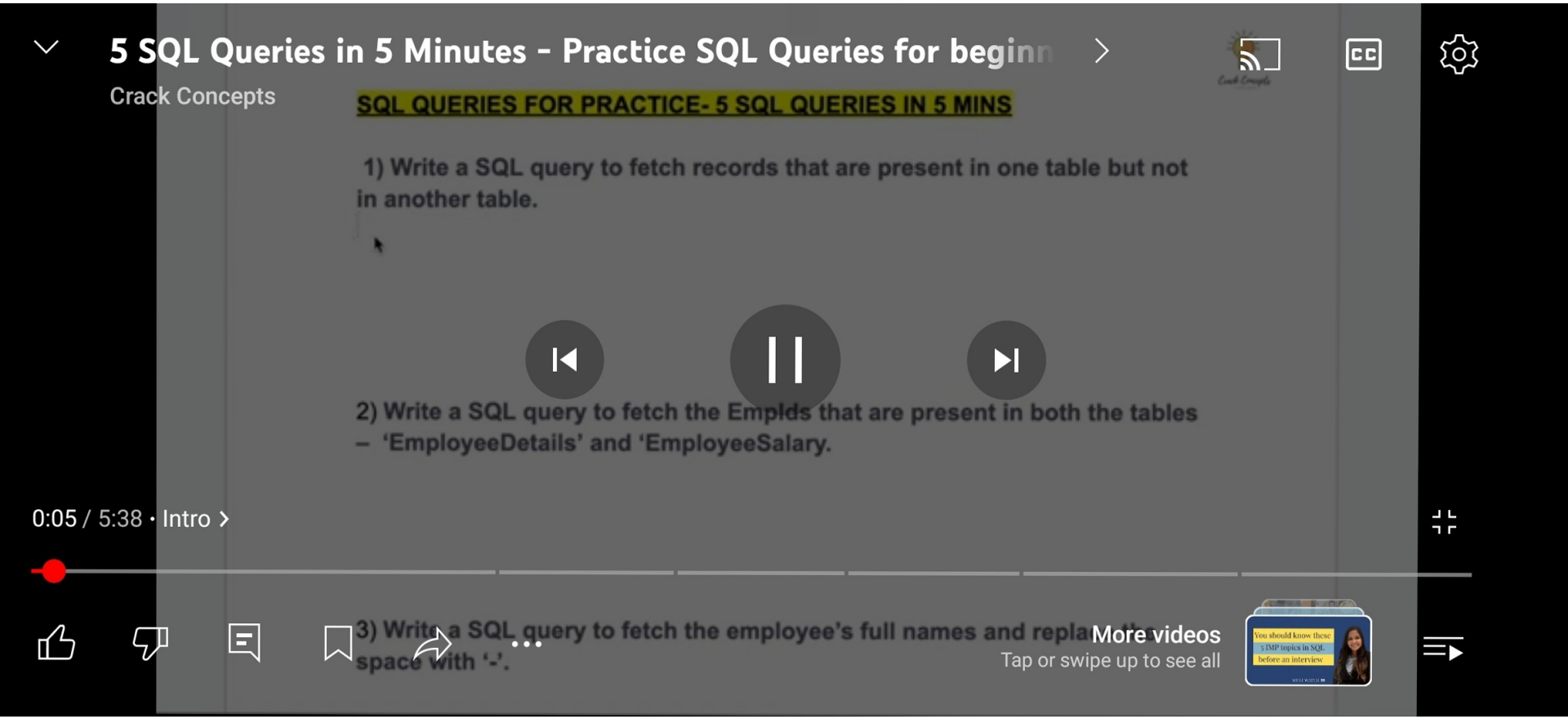
o/p:

id	name	id	marks
102	pqr	102	100
103	xyz	103	70

→ joins based on same attribute name & datatype
→ will keep only one copy of each common column

→ joins on the basis of the column explicitly specified.
→ Resulting table will contain all columns
→ BETTER
→ no ambiguity, more intuitive

Paperkraft


5 SQL Queries in 5 Minutes - Practice SQL Queries for beginn
Crack Concepts
SQL QUERIES FOR PRACTICE- 5 SQL QUERIES IN 5 MINS
1) Write a SQL query to fetch records that are present in one table but not in another table.
2) Write a SQL query to fetch the EmpIds that are present in both the tables – 'EmployeeDetails' and 'EmployeeSalary.
0:05 / 5:38 · Intro
3) Write a SQL query to fetch the employee's full names and replace the space with '-'.
More videos
Tap or swipe up to see all
You should know these 5 IMP topics in SQL before an interview

SQL QUERIES FOR PRACTICE- 5 SQL QUERIES IN 5 MINS



1) Write a SQL query to fetch records that are present in one table but not in another table.

```
SELECT * FROM TABLE1

MINUS SELECT * FROM TABLE 2

----------

SELECT E.*

FROM EMPLOYEE E

LEFT JOIN MANAGER M

USING(EMPID)

WHERE M.EMPID IS NULL
```

2) Write a SQL query to fetch the EmpIds that are present in both the tables – 'EmployeeDetails' and 'EmployeeSalary.

–SUBQUERRY

```
SELECT EMPID FROM EMPLOYEEDETAILS
WHERE EMPID IN
(SELECT EMPID FROM EMPLOYEESALARY)
```

```
Select INSTR(FullName,'hello')
From employeedetails,
```



5)Write a SQL query to uppercase the name of the employee and lowercase the city values.

```
Select UPPER(fullName), LOWER(City)
From employeedetails;
```

3) Write a SQL query to fetch the employee's full names and replace the space with '-'.

```
SELECT REPLACE(fullName,' ','-')
From employeedetails
```

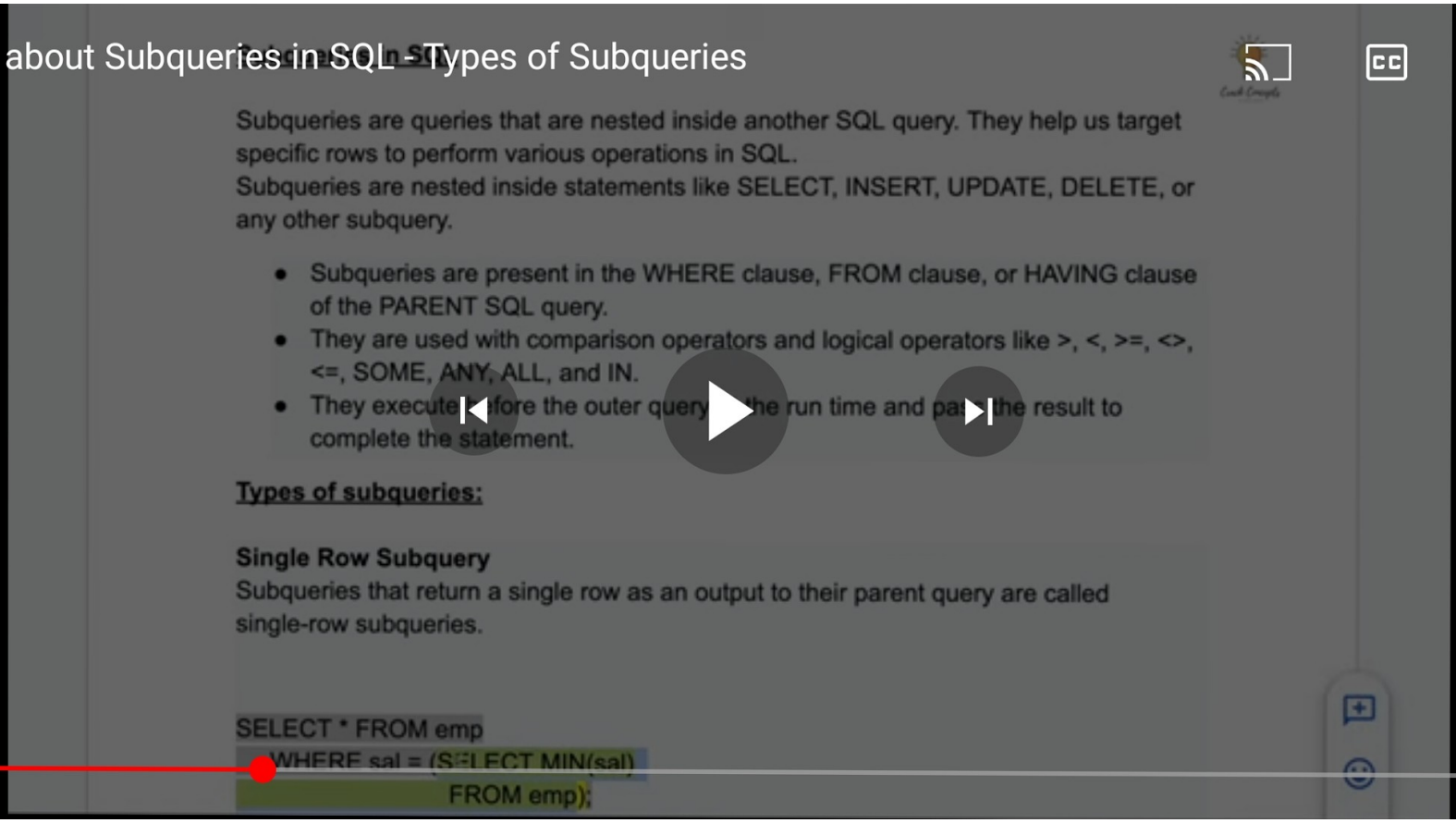


4)Write a SQL query to fetch the position of a given character(s) in a field.

```
Select INSTR(FullName,'hello')
From employeedetails,
```

5)Write a SQL query to uppercase the name of the employee and lowercase the city values.

All about Subqueries in SQL - Types of Subqueries



2:04 / 8:31

Subqueries in SQL



Subqueries are queries that are nested inside another SQL query. They help us target specific rows to perform various operations in SQL.
Subqueries are nested inside statements like SELECT, INSERT, UPDATE, DELETE, or any other subquery.

- Subqueries are present in the WHERE clause, FROM clause, or HAVING clause of the PARENT SQL query.
- They are used with comparison operators and logical operators like >, <, >=, <>, <=, SOME, ANY, ALL, and IN.
- They execute before the outer query at the run time and pass the result to complete the statement.

Types of subqueries:

Single Row Subquery

Subqueries that return a single row as an output to their parent query are called single-row subqueries.

```
SELECT * FROM emp
    WHERE sal = (SELECT MIN(sal)
                 FROM emp);
```

```
FROM paintings;
```

Multiple Row Subquery

Subqueries that return multiple rows as an output to their parent query are called multiple-row subqueries.

```
SELECT AVG(agency_fee)
FROM sales_agents
WHERE id NOT IN (SELECT id
                 FROM managers);
```

Multiple Column Subqueries

Subqueries that return multiple columns as an output to their parent query are called multiple-column subqueries.

Multiple Column Subqueries
Subqueries that return multiple columns as an output to their parent query are called multiple-column subqueries.

```
SELECT id, name, price
FROM paintings
WHERE (name, price) IN
(SELECT name, MIN(price)
FROM paintings);
```

Correlated Subqueries
Subqueries that return multiple columns as output depending on the information obtained from the parent query are called correlated subqueries.

```
SELECT last_name, salary, department_id
```

```
SELECT id, name, price
FROM paintings
WHERE (name, price) IN
(SELECT name, MIN(price)
FROM paintings);
```

Correlated Subqueries

Subqueries that return multiple columns as output depending on the information obtained from the parent query are called correlated subqueries.

```
SELECT last_name, salary, department_id
FROM employees AS outer
 WHERE salary >
          (SELECT AVG(salary)
           FROM employees
           WHERE department_id =
                 outer.department_id
            group by department_id);
```

Generally, JOINS performs faster than subqueries, but you can use them if you find it more intuitive.

Nested Subqueries

Subqueries that are inside another subquery are called nested subqueries. Subqueries are executed level by level. The innermost is executed first, and then the outer ones.

```
SELECT name AS painting,
    price,
    (SELECT AVG(price)
  FROM paintings
  WHERE price IN
     (SELECT price
      FROM paintings
      WHERE price >= 5000)
  ) AS avg_price
FROM paintings;
```

Ex for Nested subquery (cd)

```
Select name from EMP
  where Sal > (Selet avg(sal) from EMP
    where sal IN (selelt sal from EMP
      where sal > 1000.)) ;
```

↓ inner query executed 1st

order by is used inorder to sort o/p either in ASC| DESC

groupby used when we want to group our o/p.

P BY with Multiple Columns | Importan

DIFFERENCE BETWEEN

ORDER BY x	GROUP BY x
ORDER BY x, y, z	GROUP BY x, y, z
ORDER BY x, y	GROUP BY x, y

```
SELECT * FROM Customers
ORDER BY country;
```

```
SELECT x FROM subjects
GROUP BY subject;
```

```
SELECT * FROM customers
ORDER BY County, State;
```

```
SELECT * FROM subjects
GROUP BY subject, semester;
```

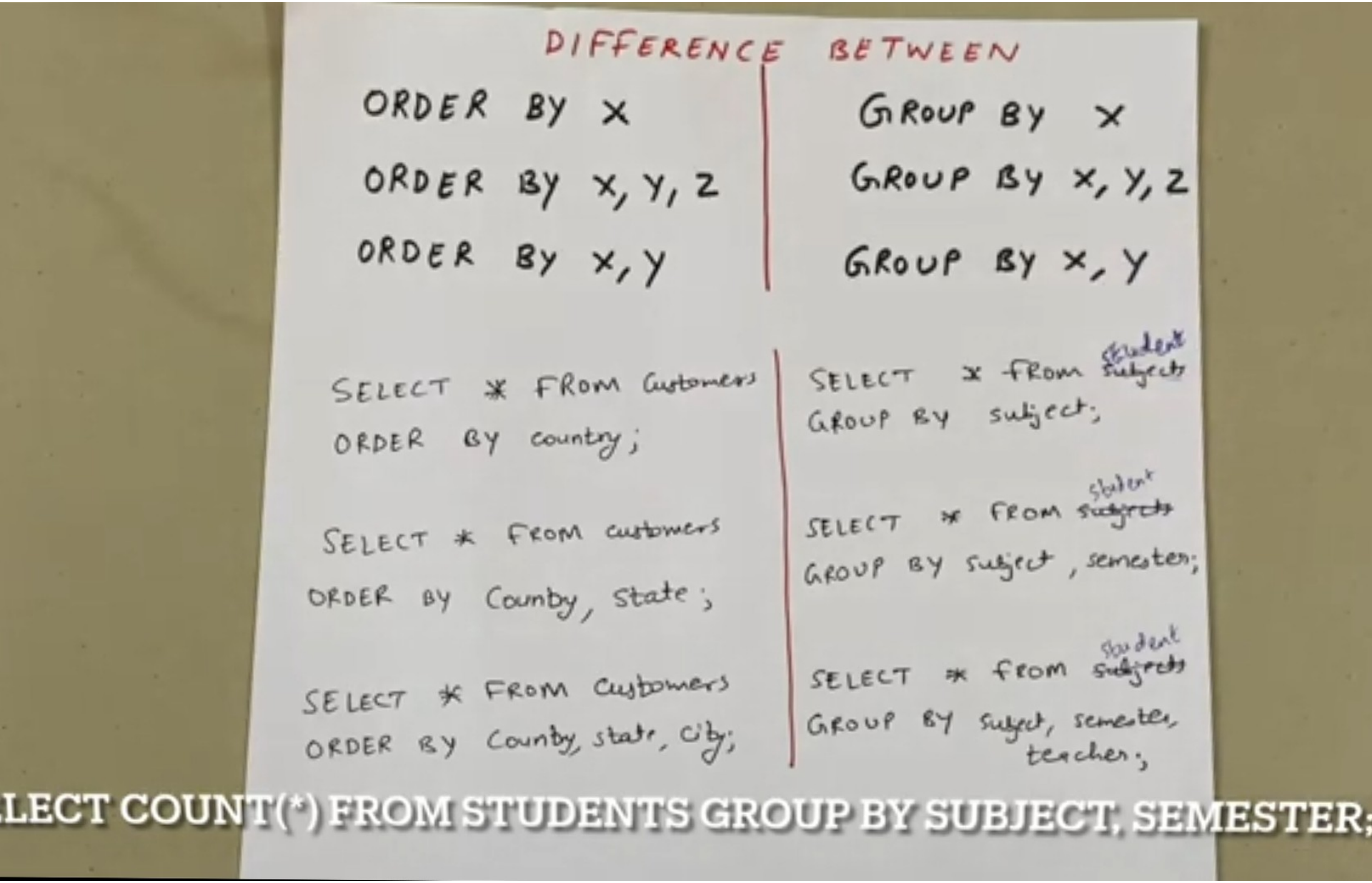
```
SELECT * FROM Customers
ORDER BY County, state, City;
```

```
SELECT * FROM subjects
GROUP BY subject, semester,
teacher;
```

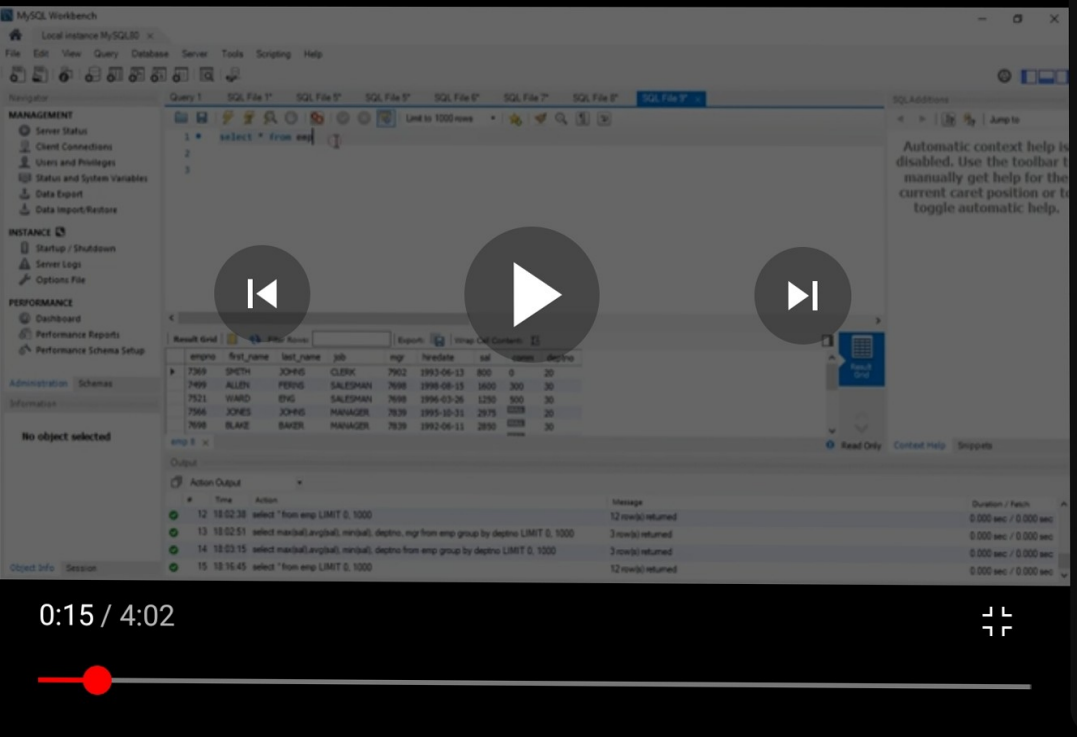
CONCEPTS

ORDER BY X,Y,Z.

FIRST ORDER BY X THEN ORDER BY Y AND THEN ORDER BY Z



LECT COUNT(*) FROM STUDENTS GROUP BY SUBJECT, SEMESTER;



Description

Query To Find highest, lowest and Average Salary In Each Department | SQL Interview Question

375	24,525	2021
Likes	Views	31 Dec

Products

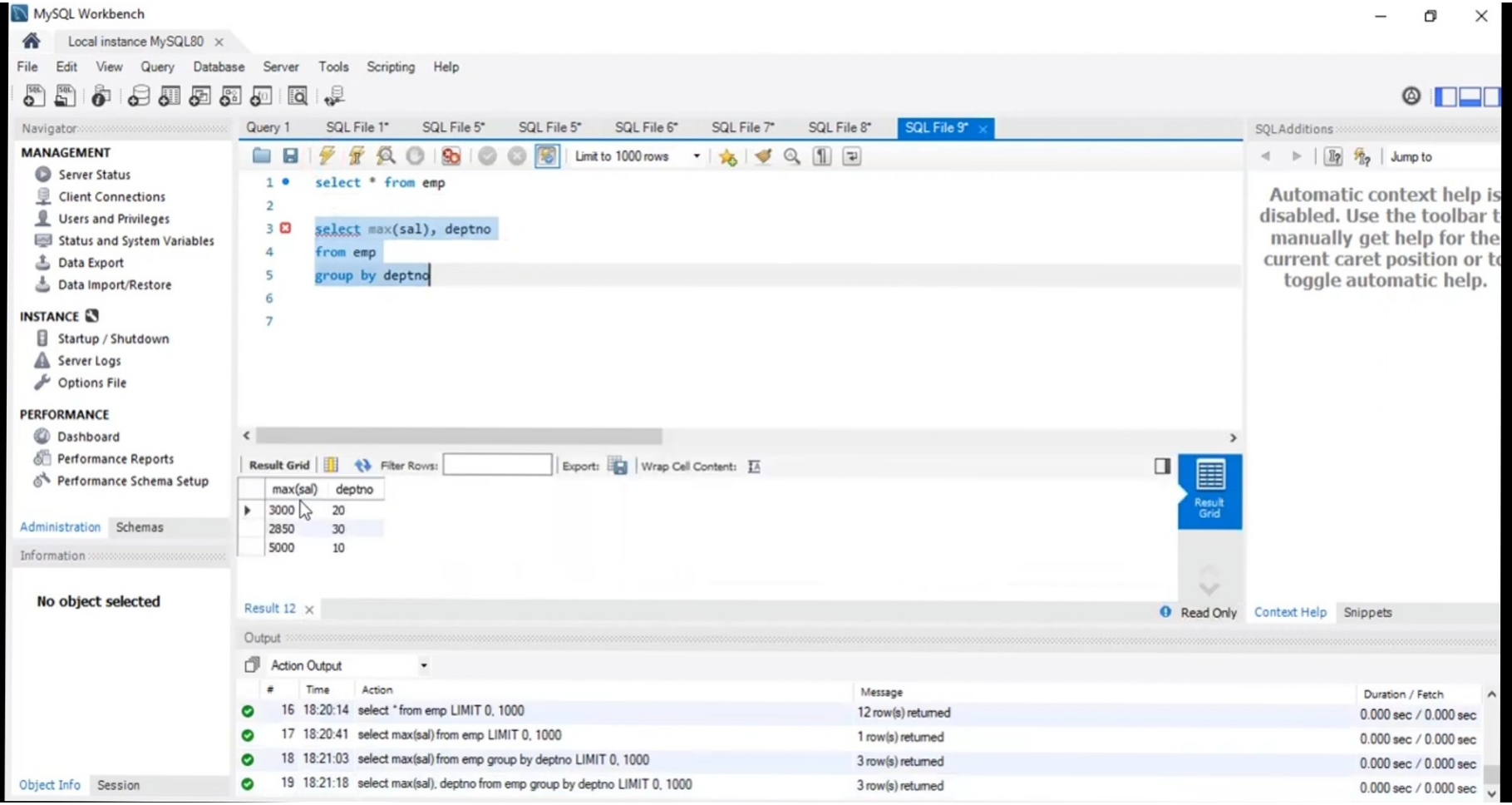
Latest from Crack Concepts shop. Learn more

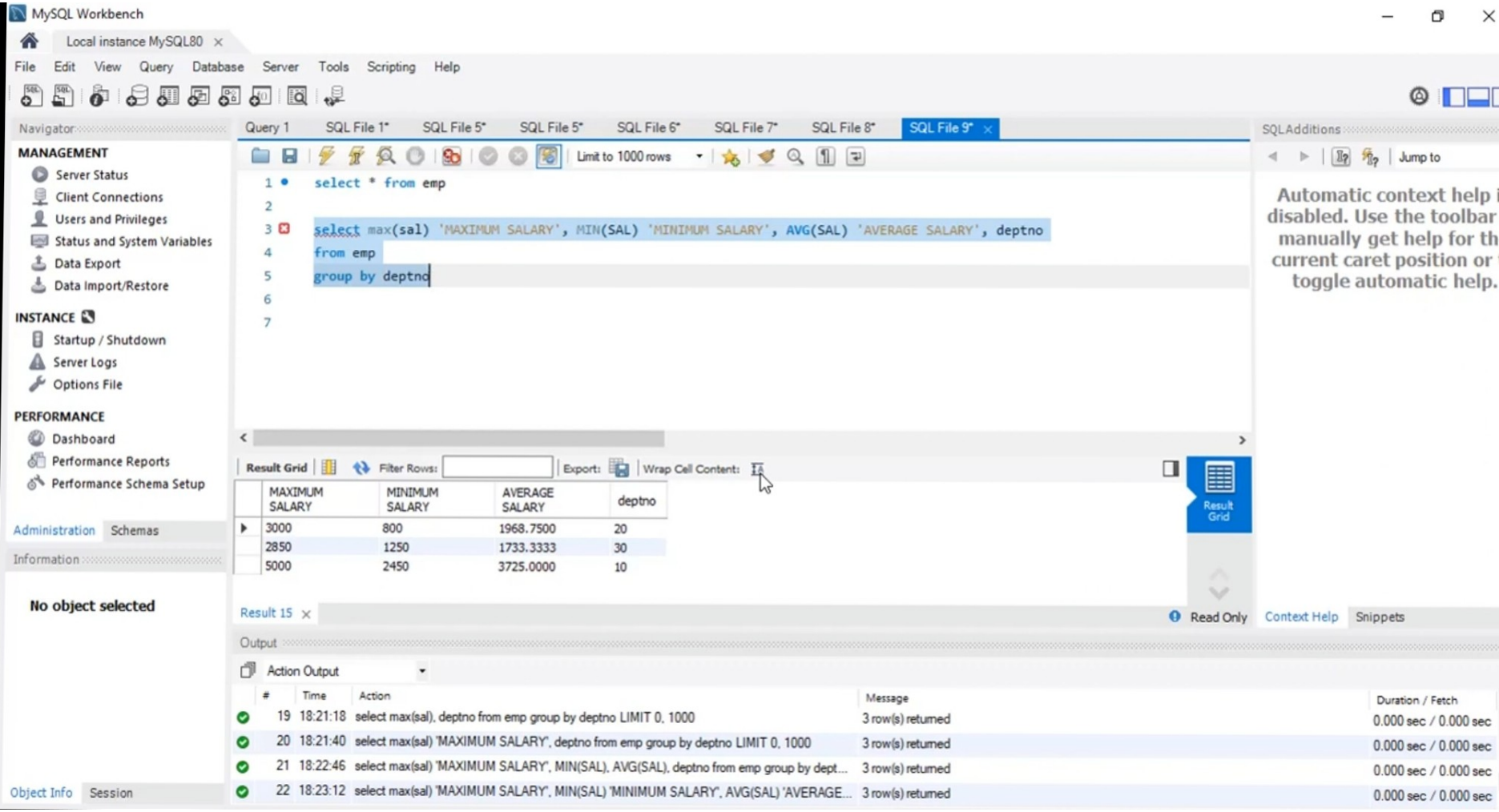


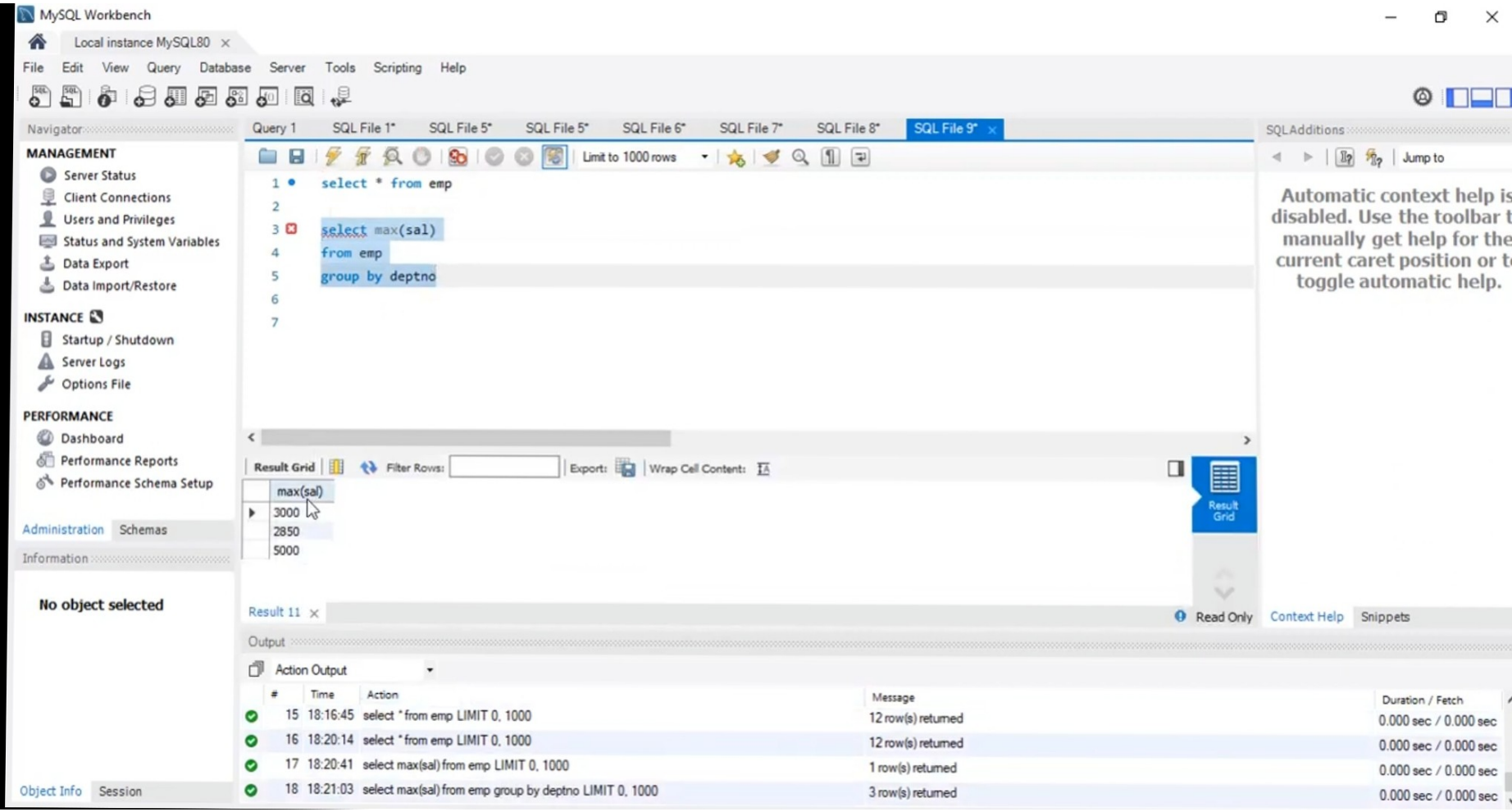
Code Coffee Repeat - Con... ₹1,468.31
crackconcepts.myspreadshop.com/...

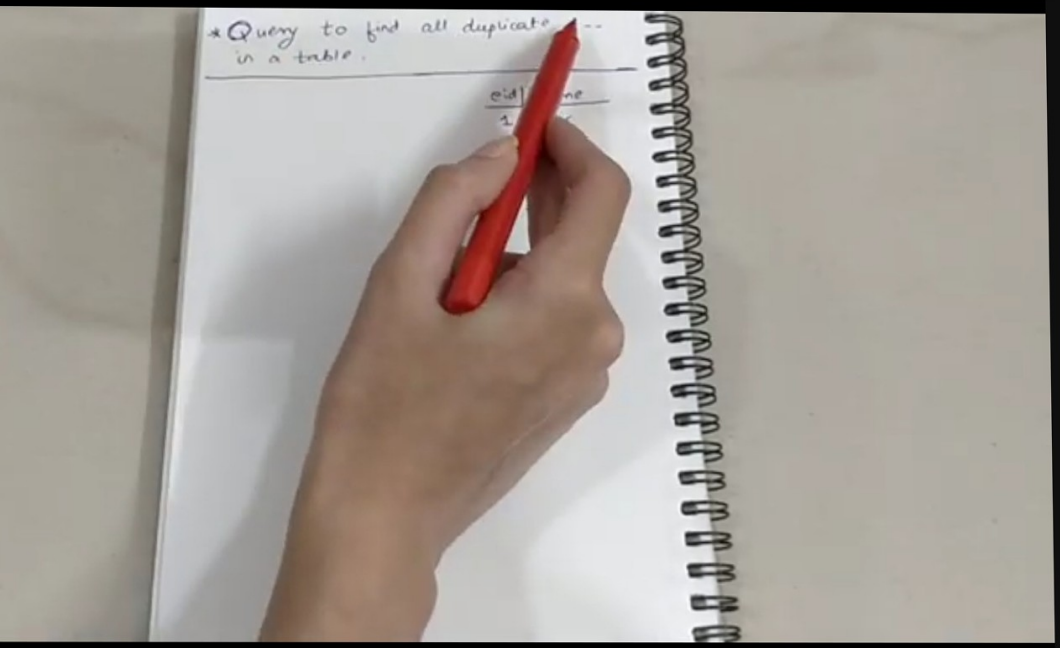


Data Nerd Merch - Contra... ₹1,468.31
crackconcepts.myspreadshop.com/


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup
Administration Schemas
Information
No object selected
Object Info Session
Query 1 SQL File 1* SQL File 5* SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9*
Limit to 1000 rows
1 select * from emp
3 select max(sal), deptno
4 from emp
5 group by deptno
Result Grid Filter Rows: Export: Wrap Cell Content:
max(sal) deptno
3000 20
2850 30
5000 10
Result 12
Read Only
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
16 18:20:14 select * from emp LIMIT 0, 1000 12 row(s) returned 0.000 sec / 0.000 sec
17 18:20:41 select max(sal) from emp LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
18 18:21:03 select max(sal) from emp group by deptno LIMIT 0, 1000 3 row(s) returned 0.000 sec / 0.000 sec
19 18:21:18 select max(sal), deptno from emp group by deptno LIMIT 0, 1000 3 row(s) returned 0.000 sec / 0.000 sec


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup
Administration Schemas
Information
No object selected
Object Info Session
Query 1 SQL File 1* SQL File 5* SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9*
Limit to 1000 rows
1 select * from emp
2
3 select max(sal) 'MAXIMUM SALARY', MIN(SAL) 'MINIMUM SALARY', AVG(SAL) 'AVERAGE SALARY', deptno
4 from emp
5 group by deptno
6
7
Result Grid Filter Rows: Export: Wrap Cell Content:
MAXIMUM SALARY | MINIMUM SALARY | AVERAGE SALARY | deptno
3000 | 800 | 1968.7500 | 20
2850 | 1250 | 1733.3333 | 30
5000 | 2450 | 3725.0000 | 10
Result 15
Read Only
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
19 18:21:18 select max(sal), deptno from emp group by deptno LIMIT 0, 1000 3 row(s) returned 0.000 sec / 0.000 sec
20 18:21:40 select max(sal) 'MAXIMUM SALARY', deptno from emp group by deptno LIMIT 0, 1000 3 row(s) returned 0.000 sec / 0.000 sec
21 18:22:46 select max(sal) 'MAXIMUM SALARY', MIN(SAL), AVG(SAL), deptno from emp group by dept... 3 row(s) returned 0.000 sec / 0.000 sec
22 18:23:12 select max(sal) 'MAXIMUM SALARY', MIN(SAL) 'MINIMUM SALARY', AVG(SAL) 'AVERAGE... 3 row(s) returned 0.000 sec / 0.000 sec


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup
Administration Schemas
Information
No object selected
Object Info Session
Query 1 SQL File 1* SQL File 5* SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9*
Limit to 1000 rows
1 select * from emp
2
3 select max(sal)
4 from emp
5 group by deptno
6
7
Result Grid Filter Rows: Export: Wrap Cell Content:
max(sal)
3000
2850
5000
Result 11
Read Only
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
15 18:16:45 select * from emp LIMIT 0, 1000 12 row(s) returned 0.000 sec / 0.000 sec
16 18:20:14 select * from emp LIMIT 0, 1000 12 row(s) returned 0.000 sec / 0.000 sec
17 18:20:41 select max(sal) from emp LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
18 18:21:03 select max(sal) from emp group by deptno LIMIT 0, 1000 3 row(s) returned 0.000 sec / 0.000 sec



Description

Query to find Duplicate Records in Table in SQL (NO DISTINCT)

2.7K	137,590	2021
Likes	Views	7 Jan

Query to find duplicate values in SQL.

If you need personal help in SQL, Mock interviews, You can contact me on work .sadiasiddiqui@gmail.com, Please note it will be a paid.

* Query to find all duplicate --- in a table.

eid	name
1	abc
2	pqr
1	abc
3	xyz
2	pqr

```
SELECT eid
FROM employee
GROUP BY eid
HAVING COUNT(eid) > 1
```

1, abc
1 abc
2 pqr
2
3 xyz

O/P 1
2

VIEWS IN SQL.

→ Virtual table.

→ by selecting fields from one or more tables present in DB. (can have conditions)

```
CREATE VIEW name AS
SELECT columns, ...
FROM table_name
WHERE condition.
```

- Single table

```
CREATE VIEW Details AS
SELECT name, address
FROM Students
WHERE id < 118;
```

ORDER BY...

- multiple tables

```
CREATE VIEW Details AS
SELECT s.name, s.address, m.marks
```

VIEWS
WHY V

3.7

Likes

This vid
views in
Proper
conveni

VIEWS IN SQL.

→ Virtual table.

→ by selecting fields from one or more tables present in DB. (can have conditions)

```
CREATE VIEW name AS
SELECT column1, ...
FROM table_name
WHERE condition.
```

- **Single table**

```
CREATE VI      Details AS
SELECT nar     addres
FROM Stud
WHERE          18;
```

ORDER BY..

- **mul**

```
      Details AS
                m.marks
```

```
CREATE VIEW name AS
SELECT column1, ...
FROM table_name
WHERE condition.
```

- **Single table**

```
CREATE VIEW Details AS
SELECT name, addres
FROM students
WHERE id < 118;
```

ORDER BY...

- **multiple tables**

```
CREATE VIEW Details AS
SELECT s.name, s.address, m.marks
FROM students s, marks m
WHERE s.id = m.id;
```

- `DROP VIEW details;`

Benefits:-

1. Views can hide complexity. (joins)
2. Views can be used as a security mechonism.

CREATE VIEW ...
SELECT name, address
FROM Students
WHERE id < 118;

ORDER BY...

- multiple tables

```
CREATE VIEW Details AS
SELECT s.name, s.address, m.marks
FROM students s, marks m
WHERE s.id = m.id;
```

- DROP VIEW details;

Benefits:-

1. Views can hide complexity. (joins)
2. Views can be used as a security mechonism.

Note:-

1. There is no record in the view, it only holds the definition of table and fetches data from the table and shows it
2. Views are stored as permanent query objects in DB.


Keys in RDBMS
Neso Academy
CS-25 PLACEMENT UPDATES
Database Management Systems
0:10 / 22:49 • Intro
Keys in RDBMS
nesoacademy.org
More videos
Tap or swipe up to see all
Keys in RDBMS (Solved Questions)
Set of 7 Questions
20 DBMS

Need for keys in RDBMS

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Keys in RDBMS

- ☆ Uniquely identify the tuple.
- ☆ Super key.
- ☆ Candidate key.
- ☆ Primary key.
- ☆ Alternate key.
- ☆ Unique key.
- ☆ Composite key.
- ☆ Foreign key.

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Super key

- ☆ Like superset.
- ☆ Uniquely identify the tuple.
- ☆ NULL values.
- ☆ {Name} is not a super key.
- ☆ May contain extraneous attributes.
- ☆ Superkeys:

  {ID}, {SSN}, {ID, Name},
  {ID, SSN}, {ID, Phone},
  {Name, Phone}, {ID, Email},
  {Name, SSN, Phone},
  {Name, Email},
  {ID, SSN, Phone} . . . . .

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Candidate key

- ☆ Superkeys:
  {ID}, {SSN}, {ID, Name},
  {ID, SSN}, {ID, Phone},
  {Name, Phone}, {ID, Email},
  {Name, SSN, Phone},
  {Name, Email},
  {ID, SSN, Phone} . . . . .
- ☆ Minimal super keys are called candidate keys.
- ☆ Candidate Keys:
  {ID}, {SSN}, {Name, Phone},
  {Email}

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

nesoacademy.org

Primary key

- ☆ To denote a candidate key.
- ☆ Candidate Keys:
  {ID}, {SSN}, {Name, Phone}, {Email}
- ☆ Primary Key: {ID}
- ☆ Chosen with care by DBA.

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Primary key

- To denote a candidate key.
- Candidate Keys:
  {ID}, {SSN}, {Name, Phone},
  {Email}
- Primary Key: {ID}
- Chosen with care by DBA.
- Never or very rarely changed.
- Candidate key with NULL value is NOT the primary key.
- Primary key = UNIQUE + NOT NULL.

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Alternate key

- ☆ The candidate key other than the primary key.
- ☆ All the keys which are not primary keys.
- ☆ Candidate Keys:
  {ID}, {SSN}, {Name, Phone}, {Email}
- ☆ Primary Key: {ID}
- ☆ Alternate Keys:
  {SSN}, {Name, Phone}, {Email}

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Unique key

- ☆ Candidate Keys:
  {ID}, {SSN}, {Name, Phone},
  {Email}
- ☆ Primary Key: {ID}
- ☆ Alternate Keys:
  {SSN}, {Name, Phone}, {Email} . . .
- ☆ Unique Key: {Name, Phone} . . .
- ☆ Composite Key: {Name, Phone} . . .

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Unique key

- ☆ Candidate Keys:
  {ID}, {SSN}, {Name, Phone},
  {Email}
- ☆ Primary Key: {ID}
- ☆ Alternate Keys:
  {SSN}, {Name, Phone}, {Email} . . .
- ☆ Unique Key: {Name, Phone} . . .
- ☆ Composite Key: {Name, Phone} . . .

Employee

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Foreign key

Student

S_ID	Name	Dept_Code	Credits
101	John	101	12
102	Robin	102	14
103	Alya	103	20
104	Yusuf	104	10

Dept

Dept_Code	Dept_Name
101	CSE
102	EEE
103	ECE
104	MECH



Referential Integrity

P.K on Create table :-

Ex: Create table persons(
ID int not null,
L-name varchar(255) not null,
F-name varchar(255), Age int primary key (ID));

→ Creates P.K on ID col. when "persons" table created

To define a P.K constraint on multiple cols :-

Create table persons (
ID int not null, L-name varchar(255) not null,
F-name varchar(255), Age int, constraint PK_person
primary key (ID, Lname)); → there is only one P.K but P.K value is made up of 2 cols

on alter table :-

create P.K constraint on ID col when the table is already created.

Ex: alter table persons add primary key (ID);

to define P.K constraint on multiple cols :-

Ex: alter table persons
add constraint PK_person primary key (ID, L-name);

→ To drop P.K constraint :-

alter table persons drop primary key;

d) foreign key constraint :-

This constraint used to prevent actions that would destroy links b/w tables

→ A F.K is a field, that refers to P.K in another table.

→ The table with F.K is called child table
|| P.K || parent (or) reference table.

on create table :- (orders, persons tables)

creates a F.K on personID col when orders table is created

Ex: Create table orders(
orderID int not null,
ordernumber int not null,
personID int,
primary key (orderID),
foreign key (personID)
references persons (personID)
);

F.K constraint on multiple cols :-

personorder ← foreign key

Multiline Comment :- starts with /* & ends with */

ex:- /* =
*/
select * from customers;

(13) Having :- clause was added to sql because where keyword can't be used with aggregate funcns.

syntax:- select col_name(s) from t_name where condition group by col_name(s) having condition order by col_names

ex:- select count(CustomerID), country from customers group by country having count(CustomerID) > 5 order by count(CustomerID) DESC;

→ lists no. of customers in each country, sorted high to low countries with than 5 customerIDs

(14) constraints :-

constraints can be specified when table is created (or) after created with the alter table statement.

syntax:- create table t_name(
col1 datatype constraint,
col2 " ,
col3 " ,
);

→ to maintain accuracy & integrity of data

→ constraints are used to specify rules for data in a table

following constraints are commonly used in sql :-

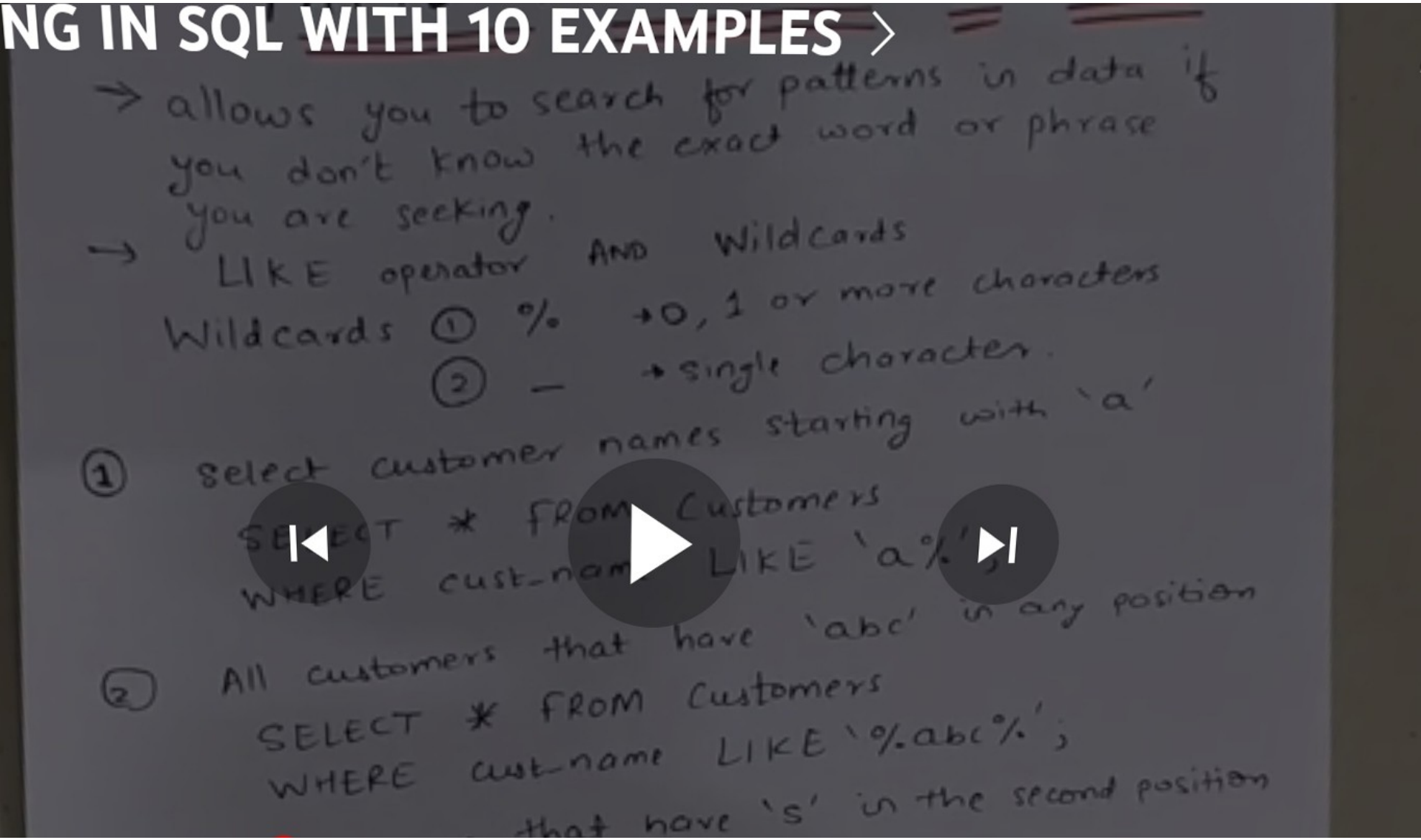
a) not null constraint :-

by default, a column can hold null values

→ not null constraint enforces a column to not accept NULL values.

→ we can't insert a new record (or) update a record without adding a value to this field.

NG IN SQL WITH 10 EXAMPLES



PATTERN MATCHING IN SQL.

→ allows you to search for patterns in data if you don't know the exact word or phrase you are seeking.

→ LIKE operator AND Wildcards

Wildcards ① % → 0, 1 or more characters
② _ → single character.

① select customer names starting with 'a'

```
SELECT * FROM Customers
WHERE cust_name LIKE 'a%';
```

② All customers that have 'abc' in any position

```
SELECT * FROM Customers
WHERE cust_name LIKE '%abc%';
```

③ All customers that have 's' in the second position

```
SELECT * FROM Customers
WHERE cust_name LIKE '_s%';
```

that start with 'a' and are

① Select customer names

```
SELECT * FROM Customers
WHERE cust_name LIKE 'a%';
```

② All customers that have 'abc' in any position

```
SELECT * FROM Customers
WHERE cust_name LIKE '%abc%';
```

③ All customers that have 's' in the second position

```
SELECT * FROM Customers
WHERE cust_name LIKE '_s%';
```

④ All customers that start with 'a' and are atleast 3 characters in length.

```
SELECT * FROM Customers
WHERE cust_name LIKE 'a__%';
```

⑤ All customers that start with 's' and end with 'a'

```
SELECT * FROM Customers
WHERE cust_name LIKE 's%a';
```

⑥ All customer names that do not start with 'a'

```
SELECT * FROM CUSTOMERS
WHERE cust_name NOT LIKE 'a%';
```

⑦ All customer names with first character is a letter in the range A through F

```
SELECT * FROM CUSTOMERS
WHERE Cust_name LIKE '[A-F]%';
```

⑧ All customer names containing exactly five characters.

```
SELECT * FROM CUSTOMERS
WHERE custname LIKE '_____';
```

⑨ All customer names that start with 'abc' or 'pqr'

```
SELECT * FROM CUSTOMERS
WHERE cust_name LIKE 'abc%'
OR cust_name LIKE 'pqr%';
```

SELECT * FROM
WHERE cust-name LIKE '[A-F]%';

(8) All customer names containing exactly five characters.

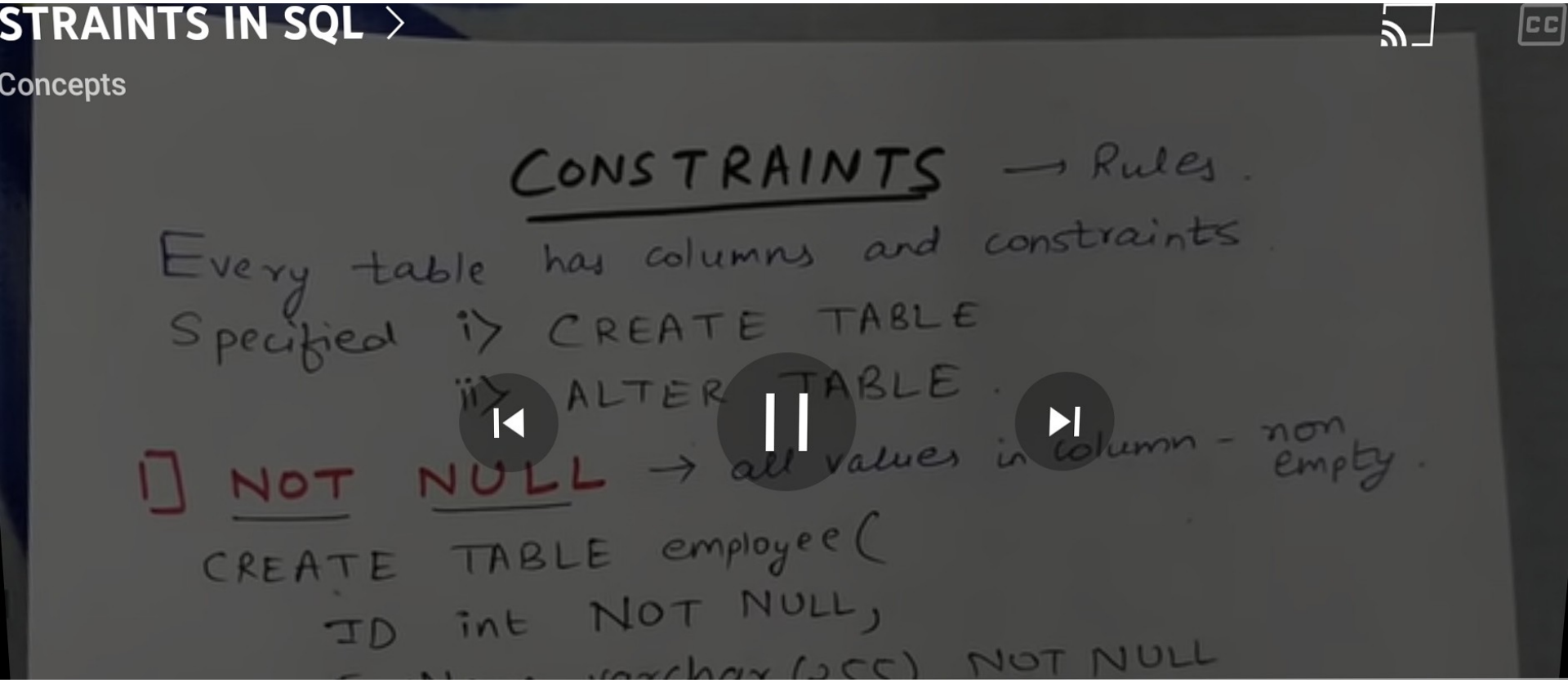
SELECT * FROM CUSTOMERS
WHERE custname LIKE '_____';

(9) All customer names that start with 'abc' or 'pqr'

SELECT * FROM CUSTOMERS
WHERE cust-name LIKE 'abc%'
OR custname LIKE 'pqr%';

(10) All customer names that end with 'th' or end with 'hg' or end with 'ri' and are of length 5.

→


STRAINTS IN SQL
Concepts
CONSTRAINTS → Rules.
Every table has columns and constraints
Specified i> CREATE TABLE
ii> ALTER TABLE.
NOT NULL → all values in column - non empty.
CREATE TABLE employee(
ID int NOT NULL,
NOT NULL

CONSTRAINTS → Rules.

Every table has columns and constraints.

Specified i> CREATE TABLE

ii> ALTER TABLE.

1] NOT NULL → all values in column - non empty.

```
CREATE TABLE employee(
    ID int NOT NULL,
    EmpName varchar(255) NOT NULL
);
```

```
ALTER TABLE Employee
MODIFY EmpName NOT NULL;
```

2] UNIQUE → all values in column - different

```
ALTER TABLE Employee
MODIFY EmpName NOT NULL;
```

2] **UNIQUE** → all values in column - different

```
ALTER TABLE employee
ADD UNIQUE (ID);
```

```
CREATE TABLE employee (
    ID int NOT NULL,
    name varchar (255)
    CONSTRAINT id_C UNIQUE (ID)
);
```

3] PRIMARY KEY

UNIQUE + NOT NULL

→ A table can have ONE primary key, but can have MANY unique constraints

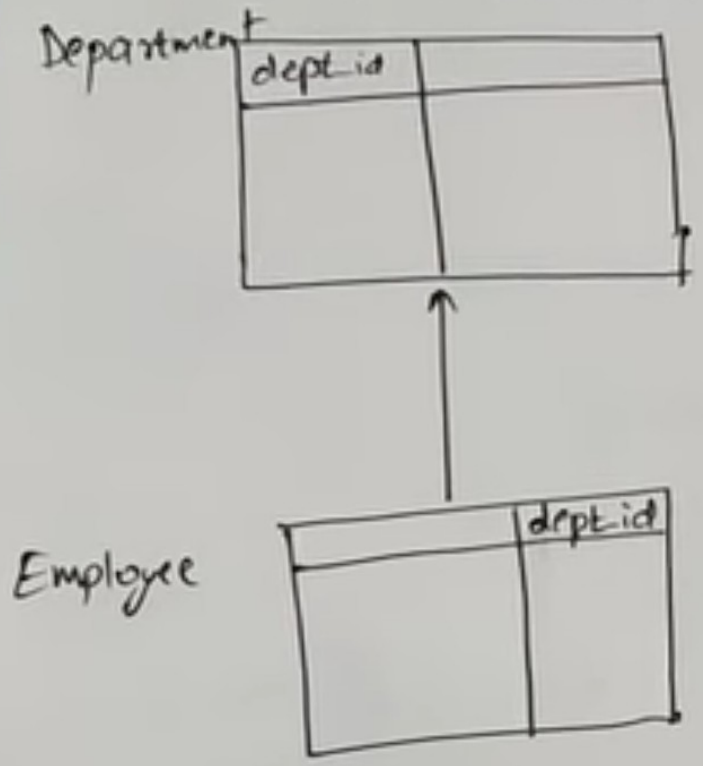
```
CREATE TABLE employee (
    ID int NOT NULL,
    name varchar (255),
    PRIMARY KEY (ID)
);
```

4] FOREIGN KEY

);

4] FOREIGN KEY

→ To maintain integrity of data.




```
CREATE TABLE Employee (
    ID int NOT NULL
    name varchar (255),
    Dept_id int,
    PRIMARY KEY (ID),
    FOREIGN KEY (Dept_id)
    REFERENCES
    DEPARTMENT (dept_id)
)
```

primary key (ID));

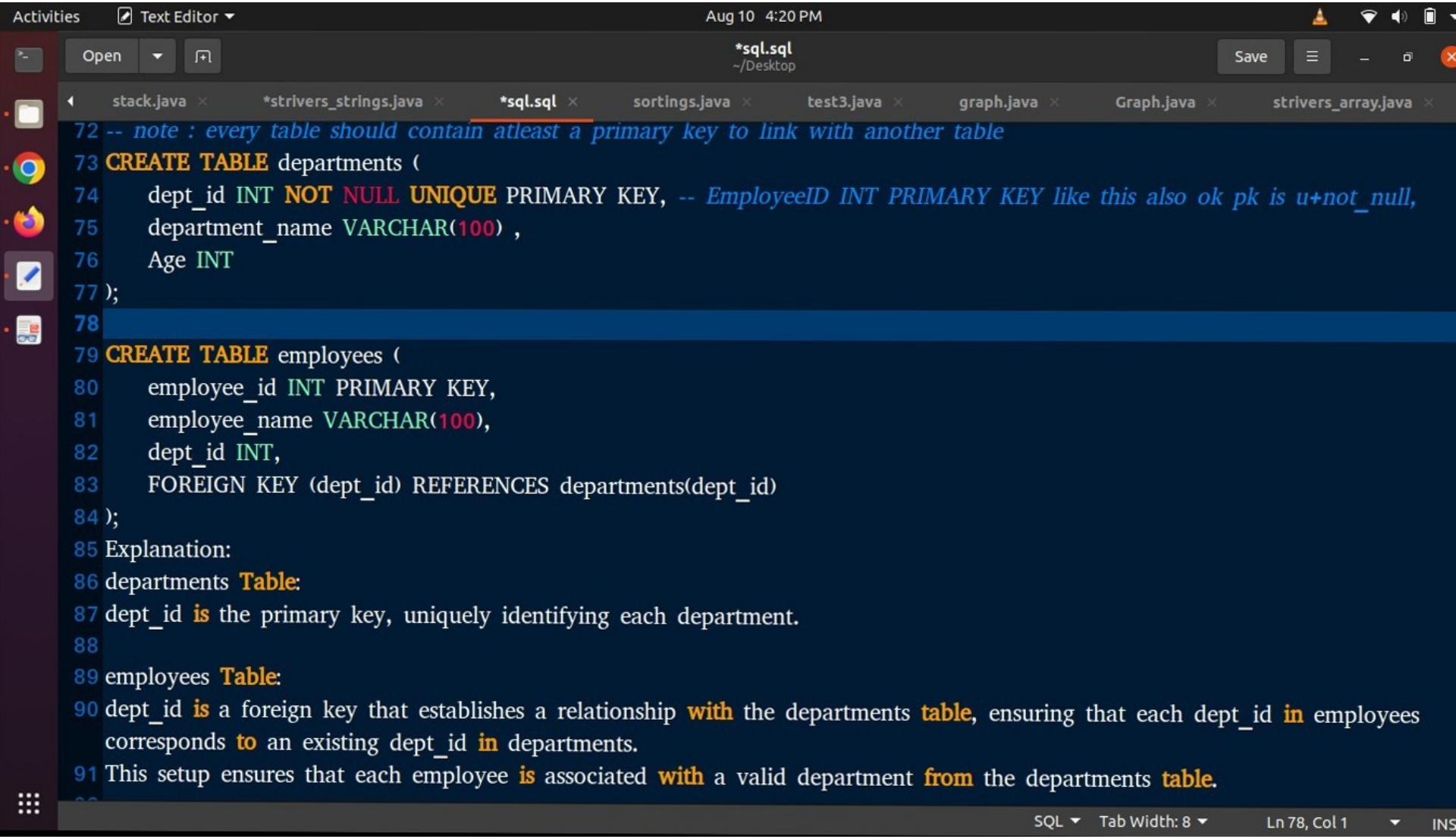
Integrity of Data using foreign key:

means we cant add / delete a value / record from both the dept-id's ie either dept-id of Department (or) dept-id of Employee becat one dept-id id. is refering to ~~each other~~ which implies the maintanance of integrity of the data in table

Check constraints:-

Syntax:- CHECK(condition)

if this condition is false then check constraint not allowed to to create / enter / alter the avalue in table


Activities
Text Editor
Aug 10 4:20 PM
Open
*sql.sql
~/Desktop
Save
stack.java
*strivers_strings.java
*sql.sql
sortings.java
test3.java
graph.java
Graph.java
strivers_array.java
-- note : every table should contain atleast a primary key to link with another table
CREATE TABLE departments (
dept_id INT NOT NULL UNIQUE PRIMARY KEY, -- EmployeeID INT PRIMARY KEY like this also ok pk is u+not_null,
department_name VARCHAR(100) ,
Age INT
);
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
employee_name VARCHAR(100),
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
Explanation:
departments Table:
dept_id is the primary key, uniquely identifying each department.
employees Table:
dept_id is a foreign key that establishes a relationship with the departments table, ensuring that each dept_id in employees corresponds to an existing dept_id in departments.
This setup ensures that each employee is associated with a valid department from the departments table.
SQL
Tab Width: 8
Ln 78, Col 1
INS

5] CHECK

```
CREATE TABLE employee (
    ID int NOT NULL,
    Age int,
    CHECK (Age >= 18)
);
```